TP 10 : Les règles de l'othello

Ce TP est à réaliser en `OCaml`.

Règles du jeu

On s'intéresse ici au jeu Othello que vous connaissez peut-être.

Il se joue sur un plateau quadrillé de taille 8×8 à l'aide de jetons ronds avec une face blanche et une face noire. Chaque joueur joue les jetons d'une certaine couleur et a pour objectif de finir la partie avec le plus grand nombre de jetons.

Lorsqu'un joueur place un jeton sur une case vide, il peut retourner des jetons de la couleur opposée selon la règle suivante : s'il encadre un ou plusieurs pions adverses entre le pion qu'il pose et un pion à sa couleur, déjà placé sur l'othellier. Cette prise en sandwich peut se faire aussi bien horizontalement ou verticalement, qu'en diagnonale. Source : Fédération française d'Othello.

La position de départ est aussi définie dans le lien précédent et peut se voir sur l'exemple suivant :

On ne peut poser un pion que si sa pose entraîne au moins un retournement. Si aucun coup n'est possible pour un joueur, il passe son tour. La partie se termine lorsqu'aucun joueur ne peut jouer. La situation où le plateau est rempli en est un cas particulier.

L'objectif des deux prochaines séances de TP est d'implémenter l'algorithme Min-Max sur un tel jeu. Le projet étant assez ambitieux, je vous propose d'avancer de manière graduelle et de commencer aujourd'hui avec la création du jeu.

Question 1 : Remporter une partie contre un ordinateur pour vous familiariser avec le jeu : ici

Types considérés

Une case peut avoir trois états différents suivant sa couleur et son occupation. On propose donc d'utiliser le type suivant :

```
type case = Blanc | Noir | Vide
```

Une configuration du jeu est entièrement décrite comme l'état de l'ensemble des cases et le joueur dont c'est le tour. On utilisera donc le type suivant :

```
type configuration =
{
    plateau : case array array;
    joueur : case;
    nombre_passe : int
}
```

L'attribut `nombre_passe` compte le nombre de tours qui ont été passés depuis le dernier jeton posé. Ainsi, lorsqu'il atteint 2, la partie se termine puisqu'aucun joueur ne peut jouer.

Enfin, on représente un coup par la position du jeton placé :

```
type coup = int*int
```

Vous devez coder dans un fichier de nom `TP10_A.ml` pour que les fichiers fournis fonctionnent avec celui-ci. Le fichier `TP10_B.ml` implémente dans des fonctions d'affichage tandis que le fichier `TP10_C.ml` est un équivalent du `main.c`.

Règles du jeu

Question 2 : Écrire une fonction `dedans : (int*int)-> bool` qui, étant donné la position d'une case, teste si celle-ci est bien sur le plateau.

Question 3 : Écrire une fonction `configuration_initiale : unit -> configuration` qui renvoie la configuration de début de jeu.

Question 4 : Écrire une fonction `trouver_direction : case array array -> case -> coup -> (int*int)-> int` qui, étant donné un plateau du jeu, un joueur, un coup et une direction, renvoie le nombre de jetons retournés dans cette direction. Une direction est un couple d'entiers parmi l'ensemble suivant :

$$\{(0,1),(1,1),(-1,1),(1,0),(-1,0),(-1,-1),(0,-1),(1,-1)\}$$

Il s'agit des directions Nord, Nord-Est, Nord-Ouest, Est, Ouest, Sud-Ouest, Sud et Sud-Est.

Question 5 : Écrire une fonction `retourner_direction : case array array -> case -> coup -> (int*int) -> bool` qui retourne le nombre maximal de jeton possibles dans la direction fournie et renvoie un booléen indiquant si au moins un jeton a été retourné.

Question 6 : Écrire une fonction `retourner : case array array -> case -> coup -> case array array* bool`. Étant donnés un plateau et un coup, renvoie le plateau obtenue après avoir joué le coup depuis le plateau en paramètre. Le booléen indiquera si des retournements de jetons ont eu lieu ou non. On fera attention à ne pas modifier l'ancienne configuration.

Question 7 : Écrire une fonction `est_legal : case array array -> case -> coup -> bool` qui, étant donné un coup, teste si le joueur dont c'est le tour est en droit de le jouer.

Question 8 : Écrire une fonction `gagnant : configuration -> case` qui, étant donné l'état du jeu, indique qui l'emporte si la partie s'arrêtait là. On renverra `Vide` si il y a égalité.

Question 9 : Écrire une fonction `coups_possibles : configuration -> configuration liste` qui étant donnée une configuration, renvoie la liste des configurations accessibles en un coup depuis celle-ci. On fera attention au cas où la liste est vide et où le joueur doit donc passer son tour.